Transitive Reduction Beyond Directed Acyclic Graphs

Antônio Drumond Cota de Sousa [**PUC Minas** | ***antonio.drumondcs@gmail.com***]
Davi Ferreira Puddo [**PUC Minas** | ***davifpuddo@gmail.com***]
Gabriel Valedo Batista Silva [**PUC Minas** | ***gabriellvbs14@gmail.com***]
João Pedro Torres [**PUC Minas** | ***torresjoaopedro00@gmail.com***]
Laura Menezes Heráclito Alves [**PUC Minas** | ***laura.heraclito@gmail.com***]
Lucas Carneiro Nassau Malta [**PUC Minas** | ***lucascarneiromalta@gmail.com***]
Mateus Henrique Medeiros Diniz [**PUC Minas** | ***mateushmdiniz@gmail.com***]
Raquel de Parde Motta [**PUC Minas** | ***apxraquel@gmail.com***]

Pontifícia Universidade Católica de Minas Gerais, R. Dom José Gaspar, 500, Coração Eucarístico, 30535-901, Belo Horizonte, MG, Brazil.

Abstract. Graph reduction techniques simplify graph representations while preserving reachability. Transitive Reduction (TR) is well defined for directed acyclic graphs, but its definition becomes ambiguous in the presence of cycles. This work revisits the notion of TR by adopting the definitions proposed by Aho *et al*. [1972], distinguishing it from the related Minimum Equivalent Graph (MEG) problem. We discuss the differences between these concepts, their computational complexity, and their applicability to cyclic, acyclic, and undirected graphs. Finally, we develop algorithms for computing these reductions under the adopted definitions.

Keywords: Graphs, Transitive Reduction, Minimum Equivalent Graph, Directed Acyclic Graph, Strongly Connected Components

1 Introduction

Graphs are abstract models that can be used to represent and help analyze many real world scenarios, such as disease analysis [Karaivanov, 2020], or data formats, such as video sequences [Belo *et al*., 2016], etc. In this field, the reachability of one vertex from another is defined by the existence of a path between them. However, multiple paths may exist between the same pair of vertices, introducing redundancy into the graph representation and making its underlying structure more difficult to analyze.

To facilitate the analysis of graphs, one may seek alternative graph representations that preserve reachability while eliminating unnecessary structural information. Such reductions can simplify subsequent graph processing tasks and have been successfully applied, for example, to temporal graph classification [Jerônimo *et al*., 2024]. One such representation is the *Transitive Reduction* (TR), which is uniquely defined for Directed Acyclic Graphs (DAGs). For general directed graphs containing cycles, however, the notion of transitive reduction becomes less straightforward, and alternative formulations are often required.

Thus, this work aims to analyze the definition of the Transitive Reduction operation, and develop efficient algorithms that perform this operation, returning an optimal result, on directed graphs. Furthermore, we will analyze how our approach handles undirected graphs, going over optimality of the result as well as computational complexity.

This paper is organized as follows. Section 2 provides the necessary background and definitions. Section 3 details the methodology. Section 4 discusses the experiments and analyzes the results. Finally, Section 5 concludes the paper.

2 Background

According to Jerônimo *et al*. [2023], the Transitive Reduction (TR) of a graph is obtained by removing all edges that are not necessary to preserve the reachability relation of the original graph. For Directed Acyclic Graphs (DAGs), this characterization is unambiguous, because, as shown by Aho *et al*. [1972], every DAG has a unique transitive reduction. Consequently, the order in which redundant edges are removed cannot affect the final result.

However, the problem becomes considerably more complex for directed cyclic graphs (DCGs), where the transitive reduction is no longer uniquely determined. The literature offers several definitions of what its result should be, and the divergence stems from conflicting notions of minimality. One line of work requires the result to be a subgraph of the original graph, admitting only the solution through the removal of edges. Another line allows the result to contain edges absent from the original graph. Such additional edges may be necessary because a strongly connected component need not already possess directed edges arranged so as to connect its vertices through a path of minimum length. Addressing this ambiguity, Aho *et al*. [1972] introduced two related but distinct concepts: the Minimum Equivalent Graph (MEG) and the Transitive Reduction (TR). The primary distinction between them is that a Minimum Equivalent Graph must be a subgraph of the original graph, whereas a Transitive Reduction is not subject to this restriction. Throughout this paper, we will adopt these definitions, which will be explained thoroughly in the remainder of this section.

2.1 Minimum Equivalent Graphs

According to Martello and Toth [1982], finding a Minimum Equivalent Graph $G' = (V, E')$ of a directed graph $G = (V, E)$ consists in removing the maximum number of edges from G while preserving its reachability. The reachability constraint requires that whenever a path connects two vertices in G, a path connecting them must also exist in G'. The edge removal constraint requires that $E' \subseteq E$. Consequently, the MEG is by definition a subgraph of G containing the fewest edges possible whilst satisfying both constraints.

2.2 Transitive Reduction

Aho *et al*. [1972] defines the Transitive Reduction (TR) of a directed graph (G) as any graph (H) with the same transitive closure and the minimum possible number of edges. Unlike a Minimum Equivalent Graph, (H) is not required to be a subgraph of (G), allowing edges absent from the original graph to be introduced when advantageous. For directed acyclic graphs, this distinction is immaterial: the unique transitive reduction is also a subgraph of (G) and is obtained by removing every edge for which an alternative directed path exists. For cyclic graphs, however, this additional flexibility allows the transitive reduction to be computed in polynomial time, avoiding the NP-complete optimization problem associated with Minimum Equivalent Graphs.

2.3 Undirected Graphs

Unlike directed graphs, reachability in an undirected graph is symmetric and therefore coincides with connectivity. As a result, preserving the reachability relation is equivalent to preserving the connected components of the graph.

Under this interpretation, the reduction problem consists of finding a minimum connectivity-preserving subgraph. For each connected component, the minimum such subgraph is a spanning tree, and therefore the reduction of an arbitrary undirected graph is a spanning forest [Diestel, 2017]. Consequently, the notions of Minimum Equivalent Graph and Transitive Reduction coincide in the undirected setting, since the resulting spanning forest is both a subgraph of the original graph and a graph with the minimum number of edges that preserves connectivity.

3 Methodology

Across all three cases (DAGs, DCGs and undirected graphs), we represent the graph with adjacency lists, mapping each vertex to a hash set of its successors. This representation uses $O(V + E)$ space and lets each breadth-first search traverse the graph in $O(V + E)$ time.

3.1 Directed Acyclic Graphs

The main algorithm removes redundant edges identified along a modified Breadth First Search (BFS). An edge that connects vertex u to vertex v, is redundant when v remains reachable from u through another path, that is, when the transitivity of the remaining edges already provides the connection it expresses. To detect this, for each edge we run a modified BFS from u that ignores the direct edge (u, v). If the search reaches v, we delete the edge. We visit every edge once and apply the test, as shown in Algorithm 1.

Algorithm 1 Redundant-edge removal by traversal

Require: directed graph G
Ensure: G without redundant edges
1: **for all** edge $(u, v) \in E(G)$ **do**
▷Returns true if there is an alternative path
2: **if** modified_BFS(G, u, v) **then**
3: remove edge (u, v) from G
4: **end if**
5: **end for**

To determine the redundancy of a single edge, the modified BFS explores the graph's adjacency lists. In the worst-case scenario, this search traverses all available vertices and edges, requiring $O(V + E)$ time. Because the algorithm systematically iterates over every edge in the initial graph to apply this test, the overall time complexity of the procedure is bounded by $O(E \times (V + E))$. On an acyclic graph, this polynomial-time procedure yields the transitive reduction, which is unique in this case and coincides with the Minimum Equivalent Graph (MEG) of G.

3.2 Directed Cyclic Graphs

Recall that a transitive reduction G' of G preserves the reachability of G with the fewest possible edges and, under the definition we adopt from Aho *et al*. [1972], it is not required to be a subgraph of G. Algorithm 1 only deletes edges, so its output is always a subgraph of the input. For an acyclic graph, both definitions are satisfied. However, when applied to a directed cyclic graph, the algorithm halts at a reachability-equivalent subgraph from which no further edge can be removed, but it is not necessarily the one with the fewest possible edges.

The limitation of Algorithm 1 in cyclic directed graphs stems from its greedy edge-removal order dependency within Strongly Connected Components (SCCs). Because the underlying redundancy test evaluates reachability unidirectionally and locally, premature edge deletions can eliminate paths crucial for the global optimum.

This sub-optimality is illustrated by a simple counter-example. Consider a strongly connected directed graph $G = (V, E)$, where the vertex set is $V = \{1, 2, 3, 4\}$ and the initial edge set is defined as: $E = \{(1, 2), (2, 1), (1, 3), (3, 1), (1, 4), (4, 1), (2, 3), (3, 4), (4, 2)\}$ as illustrated in Figure 1.

The optimal reduction that preserves all-pairs reachability corresponds to a Hamiltonian cycle containing 4 edges:

$$E_{\text{opt}} = \{(1, 2), (2, 3), (3, 4), (4, 1)\}$$

which yields the minimum equivalent graph depicted in Figure 2a.

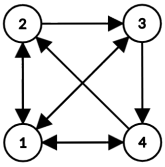


Figure 1. The original strongly connected directed graph G with 9 edges, where all vertices are mutually reachable.

However, if Algorithm 1 sequentially evaluates and removes the edges $(1, 2)$, $(1, 3)$, $(2, 1)$, and $(3, 1)$, each being a valid deletion at its respective execution step due to the temporary existence of alternative paths, the remaining subgraph consists of 5 edges:

$$E_{\text{minimal}} = \{(1, 4), (4, 1), (2, 3), (3, 4), (4, 2)\}$$

As shown in Figure 2b, this result is minimal but non-minimum. At this stage, no further edges can be removed without disrupting the strongly connected topology of the component.

(a) Minimum (Optimal) **(b)** Minimal (Local optimum)

Figure 2. Comparison of reduction outcomes. (a) The optimal Hamiltonian cycle (4 edges). (b) A minimal equivalent graph (5 edges) resulting from premature edge deletion by the greedy approach.

The algorithm becomes trapped in a local optimum ($|E_{\text{minimal}}| = 5 > |E_{\text{opt}}| = 4$) because it prematurely discarded edges essential to the global cycle. Consequently, achieving the absolute maximum reduction in cyclic graphs requires a conscious strategy for ordering edge removals, a task that directly reduces to the NP-hard Hamiltonian Cycle problem, as shown by Martello and Toth [1982].

Therefore, to reduce a graph with cycles we add a preprocessing step before applying Algorithm 1. Each SCC is contracted into a single super-vertex, turning G into its acyclic condensation, on which Algorithm 1 yields the transitive reduction exactly. The SCCs are identified with Tarjan's algorithm, proposed by Tarjan [1972], which runs a single depth-first search and reports each component in $O(V + E)$ time by tracking, for every vertex, the earliest reachable ancestor still on the search stack.

In the expansion step, every super-vertex is reintroduced as a synthetic simple cycle, and it is these synthetic edges that may place G' outside the subgraphs of G. A simple cycle with k edges is the structure that uses the fewest edges to keep a component of k vertices strongly connected, as proven by Aho *et al*. [1972].

This pipeline is a polynomial-time reduction from transitive reduction on a cyclic directed graph to the same problem restricted to acyclic graphs, with Algorithm 1 acting as an oracle for the acyclic case. The contraction maps an instance with cycles to an acyclic instance in $O(V + E)$ time, the oracle reduces it, and the expansion maps the oracle's output back to a valid reduction of the original graph in $O(V + E)$ time. The two transformations run in polynomial time and surround a single oracle call, so the cyclic case inherits the cost and the correctness of the acyclic algorithm.

Ultimately, this pipeline guarantees a mathematically sound transitive reduction for cyclic directed graphs while circumventing the NP-complete barrier of the exact Minimum Equivalent Graph problem. By systematically replacing complex SCCs with synthetic simple cycles, the procedure efficiently preserves all-pairs reachability. Although this structural mapping implies that the final output may contain synthetic edges, and thus is not strictly confined to being a subgraph of the original input, it successfully satisfies all formal properties of a transitive reduction within the established polynomial time bounds.

3.3 Undirected Graphs

The redundant-edge removal procedure of Algorithm 1 can also be applied to undirected graphs, although yielding a fundamentally different, and strictly optimal, outcome. In an undirected setting, mutual reachability is a symmetric equivalence relation that partitions the graph into connected components. An edge is redundant if and only if its endpoints remain connected upon its deletion, meaning the edge is part of a cycle (i.e., it is not a cut-edge or bridge).

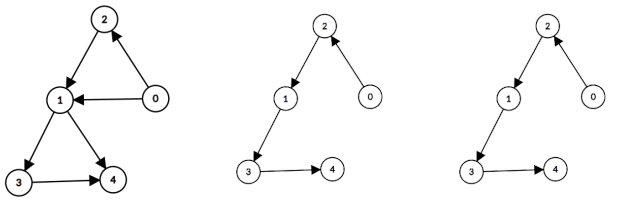
Consequently, the algorithm greedily breaks cycles by deleting one redundant edge at a time until the graph becomes entirely acyclic. This process inherently produces a collection of spanning trees (Also known as a spanning forest), one for each connected component, containing exactly $k - 1$ edges for any component of k vertices. Unlike the directed cyclic scenario, where the greedy strategy may get trapped in a local optimum, this approach on undirected graphs is guaranteed to find the absolute minimum equivalent subgraph, since the symmetric nature of undirected paths ensures that breaking cycles in any arbitrary order preserves global connectivity.

4 Experiments and Results

For the experiments involving directed acyclic graphs and undirected graphs, we evaluated our implementation against an exponential algorithm, which checked all possible combinations of edge removal and returned the smallest valid one. Our primary metric for comparison was the final edge count of the resulting graph.

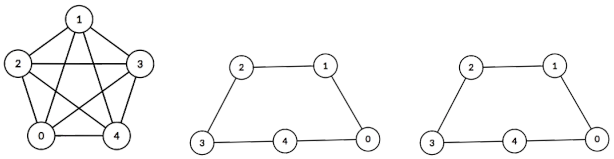
The principle of our experiment regarding directed cyclic graphs involves examples for which the answer yielded by the exponential algorithm is known to be non-optimal. We show that our implementation returns a graph that is a valid TR, achieving the same edge count if removals are enough, and fewer edges if edge additions are also necessary to reach optimality. We also contrast the number of edge checks performed by both approaches to highlight differences in efficiency.

The visual results all contain three graphs. From left to right: the first one is the input graph, the second one is our algorithm's answer, and the third one is the exponential algorithm's answer.



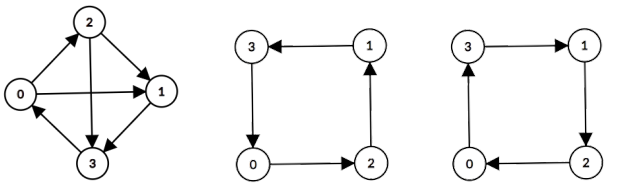
(a) Base graph **(b)** Pre-processed greedy solution **(c)** Exponential solution

Figure 3. Results in a DAG



(a) Base graph **(b)** Pre-processed greedy solution **(c)** Exponential solution

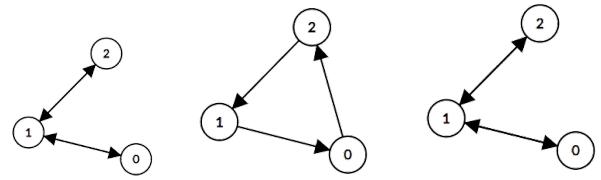
Figure 4. Results in a non-directed graph



(a) Base graph **(b)** Pre-processed greedy solution **(c)** Exponential solution

Figure 5. Results in a DCG (edge additions unnecessary)

Our algorithm found a graph with the same final edge count as the exponential algorithm for Figure 3b, Figure 4b, and Figure 5b. Since all of those instances required only edge removals to find a TR, we successfully reached the optimal solution with our proposed algorithm.



(a) Base graph **(b)** Pre-processed greedy solution **(c)** Exponential solution

Figure 6. Results in a DCG (edge additions necessary)

Analyzing Figure 6b, our algorithm found a TR with fewer edges than the exponential algorithm. This means that our algorithm optimally answered the TR problem for a counter-example of the TR algorithm without pre-processing, while also evaluating the redundant edges more efficiently during the checking phase.

5 Conclusion

In this work, we presented the problems of Transitive Reduction and Minimum Equivalent Graph, discussing their applications, definitions, and differences for cyclic and acyclic directed graphs, as well as undirected graphs. We also developed algorithms to compute these graph reductions. By adopting the definitions proposed by Aho *et al*. [1972] and highlighting the distinctions between TR and MEG, this work contributes to a clearer understanding of these closely related concepts.

Authors' Contributions

Though all authors participated in all steps of this work, the main contributions of each were the following:

- **Antônio Drumond Cota de Sousa:** Implementation of graph data structure and algorithms.
- **Davi Ferreira Puddo:** Development and execution of tests.
- **Gabriel Valedo Batista Silva:** Implementation of random graph generator.
- **João Pedro Torres:** Writing report.
- **Laura Menezes heráclito Alves:** Literature review.
- **Lucas Carneiro Nassau Malta:** Literature review.
- **Mateus Henrique Medeiros Diniz:** Development and execution of tests.
- **Raquel de Parde Motta:** Writing report.

Availability of data and materials

All code developed for this project is available on the following GitHub repository: https://github.com/AntonioDrumond/PAA

References

Aho, A. V., Garey, M. R., and Ullman, J. D. (1972). The transitive reduction of a directed graph. *SIAM Journal on Computing*, 1(2):131–137. DOI: 10.1137/0201008.

Belo, L., Caetano, C., Patrocínio Jr, Z., and Guimarães, S. (2016). Summarizing video sequence using a graph-based hierarchical approach. *Neurocomputing*, 173:1001–1016. DOI: 10.1016/j.neucom.2015.08.057.

Diestel, R. (2017). *Graph Theory*. Springer, 5 edition.

Jerônimo, C., Malinowski, S., Patrocínio, Z. K. G., Gravier, G., and Guimarães, S. J. F. (2023). *Filtering Safe Temporal Motifs in Dynamic Graphs for Dissemination Purposes*, pages 480–493. DOI: 10.1007/978-3-031-49018-7_34.

Jerônimo, C., Patrocínio Jr, Z., Malinowski, S., Gravier, G., and Guimarães, S. (2024). Decreasing graph complexity with transitive reduction to improve temporal graph classification. *International Journal of Data Science and Analytics*, 19:229–242. DOI: 10.1007/s41060-024-00632-8.

Karaivanov, A. (2020). A social network model of covid-19. *PLOS ONE*, 15(10):1–33. DOI: 10.1371/journal.pone.0240878.

Martello, S. and Toth, P. (1982). Finding a minimum equivalent graph of a digraph. *Networks*, 12(2):89–100. DOI: https://doi.org/10.1002/net.3230120202.

Tarjan, R. (1972). Depth-first search and linear graph algorithms. *SIAM journal on computing*, 1(2):146–160.